

```

import streamlit as st
import base64

st.set_page_config(page_title="ProfIA
– Intelligence pédagogique unifiée",
page_icon="◆", layout="centered")

# ===== 1. DESIGN NOIR / OR
=====
st.markdown("""
<style>
.stApp { background-color: #0A0A0B;
color: #EDEAE2; }
section[data-testid="stSidebar"] {
background-color: #131315; border-
right: 1px solid #2A2A2D; }
h1, h2, h3 { font-family: Georgia,
serif; color: #EDEAE2; }
.stChatMessage { background-color:
#1B1B1E; border-radius: 6px; }
.stButton>button {
    background-color: #C9A227; color:
#0A0A0B; border: none; font-weight:
600;
}
.stButton>button:hover { background-

```

```

color: #DDB74A; color: #0A0A0B; }
.stTextInput>div>div>input,
.stTextArea textarea,
.stSelectbox>div>div {
    background-color: #1B1B1E
!important; color: #EDEAE2
!important; border: 1px solid #2A2A2D
!important;
}
[data-testid="stChatInput"] textarea
{ background-color: #1B1B1E
!important; color: #EDEAE2
!important; }
.badge {
    display:inline-block; border:1px
solid #2A2A2D; border-radius:14px;
padding:3px 10px;
    font-size:0.75rem; color:#8C897F;
margin-right:6px;
}
</style>
""", unsafe_allow_html=True)

# ===== 2. CLES API (Secrets)
=====
def get_secret(name):
    try:
        return st.secrets[name]
    except Exception:

```

```

        return ""

    CLAUDE_KEY = get_secret("CLAUDE_KEY")
    OPENAI_KEY = get_secret("OPENAI_KEY")
    GEMINI_KEY = get_secret("GEMINI_KEY")

    CLAUDE_MODEL = "claude-haiku-4-5-20251001"
    OPENAI_MODEL = "gpt-5.6-sol"
    GEMINI_MODEL = "gemini-3.5-flash"

    # ===== 3. CONSTITUTION
    =====
    def build_system_prompt(classe,
                           matiere):
        return f""Tu es ProfIA v1.0. Le
meilleur tuteur scolaire au monde.

RÈGLES STRICTES :
1. MISSION : Faire progresser. Ne
donne JAMAIS la réponse directe.
Guide par 1 question.
2. ZONE : Scolaire/Universitaire
uniquement. Hors-sujet = "Je suis
ProfIA, je traite que les cours."
3. ADAPTATION : Détecte la classe
{classe}. CP=simple+emojis.
Master=rigueur+démo.
4. PSYCHO : Patient, encourageant. Si

```

"je suis nul" → "On bloque sur 1 étape. On la fait ensemble."
5. ANTI-TRICHE : Si "DS", "Contrôle", "Examen" → "Je ne peux pas aider pendant un examen. On révise après ?"
6. VÉRITÉ : Si tu ne sais pas → "Je ne sais pas". Zéro invention.
7. ÉTHIQUE : Zéro pub, zéro biais, données sécurisées Afrique/Europe.
8. PÉDAGOGIE : Montre toujours les étapes. Utilise des exemples locaux.
9. SÉRIEUX : Ton professionnel et posé en toute circonstance. Aucune blague, aucun trait d'humour, aucune plaisanterie – l'usage d'emojis reste réservé au cas CP prévu par la règle 3.

Matière actuelle : {matiere}""

NIVEAUX =

["CP", "CE1", "CE2", "CM1", "CM2", "6e", "5e", "4e", "3e", "2nde", "1ère", "Terminale", "Licence", "Master"]

MATIERES =

["Mathématiques", "Physique", "SVT", "Français", "Anglais", "Histoire-Géo", "Philosophie", "Informatique", "ECM", "Autre"]

```

# ===== 4. APPELS AUX 3 IA
(chacun isolé, ne peut jamais faire
planter le site) =====
def call_claude(system, history,
image_b64=None, image_mime=None):
    try:
        import anthropic
        client =
anthropic.Anthropic(api_key=CLAUDE_KEY)

        messages = []
        for m in history:
            content = []
            if m.get("image_b64"):

content.append({"type": "image", "source":
{"type": "base64", "media_type": m["image_mime"], "data": m["image_b64"]})

content.append({"type": "text", "text":
m["content"] or "Regarde la photo
jointe."})
            messages.append({"role":
m["role"], "content": content})
            resp =
client.messages.create(model=CLAUDE_MODEL, max_tokens=1024, system=system,

```

```

messages=messages)
        return "".join(b.text for b
in resp.content if hasattr(b,
"text")).strip(), None
    except Exception as e:
        return None, str(e)

def call_openai(system, history):
    try:
        import openai
        client =
openai.OpenAI(api_key=OPENAI_KEY)
        messages =
[{"role": "system", "content": system}]
        for m in history:
            if m.get("image_b64"):

messages.append({"role": m["role"],
"content": [

{"type": "text", "text": m["content"]}
or "Regarde la photo jointe."},

{"type": "image_url", "image_url":
{"url": f"data:
{m['image_mime']};base64,
{m['image_b64']}"}}
                ]})
            else:

```

```

messages.append({"role": m["role"],
"content": m["content"]})
        resp =
client.chat.completions.create(model=
OPENAI_MODEL, messages=messages,
max_tokens=1024)
        return
resp.choices[0].message.content.strip
(), None
        except Exception as e:
            return None, str(e)

def call_gemini(system, history):
    try:
        import google.generativeai as
genai

genai.configure(api_key=GEMINI_KEY)
        model =
genai.GenerativeModel(GEMINI_MODEL,
system_instruction=system)
        contents = []
        for m in history:
            parts = [m["content"] or
"Regarde la photo jointe."]
            if m.get("image_b64"):

parts.append({"mime_type":

```

```

m["image_mime"], "data":
base64.b64decode(m["image_b64"]))}
        contents.append({"role":
"model" if m["role"]=="assistant"
else "user", "parts": parts})
        resp =
model.generate_content(contents)
        return resp.text.strip(),
None
    except Exception as e:
        return None, str(e)

```

```

PROVIDERS = [
    ("claude", CLAUDE_KEY,
call_claude),
    ("openai", OPENAI_KEY,
call_openai),
    ("gemini", GEMINI_KEY,
call_gemini),
]

```

```

def fuse_drafts(system, last_text,
drafts):
    """Fusionne plusieurs brouillons
en une seule réponse, via le premier
modèle disponible."""
    fusion_system = system + """

```

Tu reçois ci-dessous plusieurs

brouillons de réponse à la même question d'élève, rédigés indépendamment. Fusionne-les en UNE seule réponse ProfIA optimale : garde le meilleur raisonnement de chaque brouillon, corrige les erreurs éventuelles, élimine les répétitions, et respecte strictement les règles ci-dessus. Ne mentionne jamais qu'il existe plusieurs brouillons ou plusieurs assistants : réponds directement en tant que ProfIA."""

```
drafts_text = "\n\n---\n\n".join(f"Brouillon {i+1} :\n{d}"
for i, d in enumerate(drafts))
fusion_history = [{"role": "user",
"content": f'Question de l\'élève : "{last_text}"\n\n{drafts_text}',
"image_b64": None, "image_mime":
None}]
for name, key, fn in PROVIDERS:
    if not key:
        continue
    if name == "claude":
        text, err =
fn(fusion_system, fusion_history)
    else:
        text, err =
fn(fusion_system, fusion_history)
```

```

        if text:
            return text
        return drafts[0] # filet de
sécurité ultime : jamais de plantage

def get_profia_answer(classe,
matiere, history):
    system =
build_system_prompt(classe, matiere)
    active = [(n, k, fn) for n, k, fn
in PROVIDERS if k]
    if not active:
        return None, "Aucune clé API
n'est configurée sur ce serveur (voir
.streamlit/secrets.toml)."

    drafts = []
    errors = []
    for name, key, fn in active:
        text, err = fn(system,
history)
        if text:
            drafts.append(text)
        else:
            errors.append(f"{name}:
{err}")

    if not drafts:
        return None, " |

```

```
".join(errors) if errors else "Aucun  
modèle n'a répondu."
```

```
    last_text = history[-1]  
    ["content"] if history else ""  
    if len(drafts) == 1:  
        return drafts[0], None  
    try:  
        return fuse_drafts(system,  
last_text, drafts), None  
    except Exception:  
        return drafts[0], None # si  
la fusion échoue, on renvoie quand  
même une réponse valable
```

```
# ===== 5. INTERFACE =====  
st.markdown("## ♦ ProfIA")  
st.caption("Intelligence pédagogique  
unifiée")
```

```
with st.sidebar:  
    st.markdown("#### Contexte")  
    classe = st.selectbox("Classe",  
NIVEAUX, index=5)  
    matiere = st.selectbox("Matière",  
MATIERES, index=0)  
    photo = st.file_uploader("Photo  
d'exercice (optionnel)", type=  
["png", "jpg", "jpeg"])
```

```

        if st.button("Nouvelle
conversation"):
            st.session_state.messages =
            []

            st.rerun()
            n_active = sum(1 for _, k, _ in
PROVIDERS if k)
            st.markdown(f"<span
class='badge'>{n_active} modèle(s)
actif(s)</span>",
unsafe_allow_html=True)

        if "messages" not in
st.session_state:
            st.session_state.messages = []

        for msg in st.session_state.messages:
            with st.chat_message("user" if
msg["role"]=="user" else
"assistant"):
                st.markdown(msg["content"])
                if msg.get("image_b64"):

st.image(base64.b64decode(msg["image_
b64"]))

prompt = st.chat_input("Écris ta
question ici...")

```

```

if prompt or photo:
    image_b64, image_mime = None,
    None
    if photo:
        raw = photo.read()
        image_b64 =
base64.b64encode(raw).decode("utf-8")
        image_mime = photo.type or
        "image/jpeg"

    user_text = prompt or "Voici une
photo de mon exercice, aide-moi."

st.session_state.messages.append({"ro
le":"user","content":user_text,"image
_b64":image_b64,"image_mime":image_mi
me})

    with st.chat_message("user"):
        st.markdown(user_text)
        if image_b64:

st.image(base64.b64decode(image_b64))

    with
st.chat_message("assistant"):
        with st.spinner("ProfIA
réfléchit..."):
            answer, error =

```

```
get_profia_answer(classe, matiere,  
st.session_state.messages)  
    if answer:  
        st.markdown(answer)  
  
st.session_state.messages.append({"ro  
le":"assistant","content":answer,"ima  
ge_b64":None,"image_mime":None})  
    else:  
        st.error(f"ProfIA n'a pas  
pu répondre pour le moment. Détail  
technique : {error}")
```